Chinese Legal Issue Triage

Multi-label NLP Classification from Everyday Legal Narratives

Course: Natural Language Processing (NLP) · Final Project **System name:** LexTag — Legal Issue Triage **Disclaimer:** This project performs legal *issue triage* for NLP education. It does **not** provide legal advice and does **not** generate legal opinions.

---

1. Research Question

Ordinary people describe legal trouble in everyday Chinese (“someone copied my photo and posted insults under my name”), but they rarely know *which legal issue* their situation belongs to. Existing Taiwanese legal-AI products (Lawsnote, Lawbot, judgment search engines) mostly start one step too late: they assume the user already knows they have a defamation problem, a contract problem, or a fraud problem.

This project asks one focused question:

> **Can an NLP classifier map a free-text Chinese legal scenario to one or more legal-issue labels — *before* any retrieval, chatbot, or legal-advice step — and surface the relevant statute text as a lookup clue?**

The deliverable is an **intake triage classifier**, not a chatbot. The output is a structured set of issue labels (e.g. `酒駕/公共危險`, `詐欺`, `妨害名譽`), confidence scores, and the **full text of the supporting statutes** as reference clues.

---

2. Practical Application

This is the part the course guideline weights most heavily (30%), so it drives the whole design.

Why this task matters. Legal question answering and RAG over judgments are crowded and depend on large proprietary databases. Issue *triage* is an unsolved, upstream bottleneck: every legal-aid hotline, law-firm intake form, and legal-information website needs to route a person’s story to the right area of law before anything else can happen.

Who benefits.

- **Legal-aid intake services** — auto-route incoming requests to the correct duty lawyer/category, reducing manual triage load.

- **Legal-information websites** — turn a free-text complaint into the right FAQ / article category.
- **Law-firm client intake** — pre-fill a structured issue tag and candidate statutes before the first consultation.
- **Law students** — practice connecting plain-language facts to legal concepts and the underlying articles.

How it solves a practical problem. Instead of competing with full legal-AI products, LexTag occupies the cheap, measurable, high-leverage first step: *classification*. Because the output is a label set (not free-text advice), the task is **measurable** (F1, recall, precision) and **auditable** — and it avoids the liability and hallucination risk of generating legal opinions. The statute-text lookup gives the user a concrete, verifiable starting point without the system ever claiming the article *applies* to their case.

---

3. Dataset Description

3.1 Sources

Dataset	Size	Role in this project
`lianghsun/tw-legal-synthetic-qa`	9,631 QA pairs	**Main weak-label source** — Chinese legal scenario (user turn) + analysis (assistant turn)
`lianghsun/tw-processed-law-article`	230,974 statute rows	**Statute lookup + ontology support** — provides the actual article text shown in the demo

The QA dataset is in ShareGPT-style message format. We treat the **user message** as the scenario to classify, and use the **assistant message only as a weak-label signal**, because the analysis usually cites specific statutes (e.g. `刑法第185條之3`).

3.2 Preprocessing pipeline

1. **Citation normalization** (`scripts/citation_normalizer.py`) — converts varied Chinese citation forms (`刑法第185條之3`, Arabic/Chinese numerals, `臺/台` variants, full-width spaces) into stable `(law, article)` keys via NFKC normalization + a regex + a Chinese-numeral parser.
2. **Ontology construction** (`data/issues.yaml`) — a hand-built ontology of **20 everyday legal issues** (12 criminal, 8 civil), each mapping to its supporting articles plus aliases and keywords.

3. **Weak labeling** (`scripts/build_labels.py`) — extract citations from each assistant answer, map them through the ontology to issue labels; keep only rows with ≥1 label and a scenario ≥20 characters.
4. **Statute-text lookup** (`scripts/build_law_lookup.py`) — for the 26 `(law, article)` pairs the ontology references, extract the **full article text** from `tw-processed-law-article` into `data/law_articles.json` (heading `<law> 第N 條` stripped, body kept). All 26 articles resolved (13 criminal, 13 civil).
5. **Deterministic split** (`scripts/split_data.py`) — 70/15/15 train/val/test with a fixed seed.

3.3 Resulting dataset

- **1,523** weakly-labeled rows → Train **1,066** / Val **228** / Test **229**.

3.4 Limitations (stated honestly)

- **Weak labels, not gold labels** — labels are derived from synthetic legal analysis, so label noise is real.
- **Severe class imbalance** — `酒駕/公共危險` alone is 672/1,523 (**44%**), while the rarest classes (`所有物返還`, `繼承`, `夫妻財產`) have only 9–11 examples each.
- **Domain bias** — criminal and civil substantive law only; procedural law is intentionally excluded (this is everyday issue triage, not court-procedure classification).
- **No guarantee of legal applicability** — a surfaced article is a lookup clue, never a claim that it governs the specific case.

---

4. NLP Method

The pipeline has five stages, mapping directly to course topics (tokenization, TF-IDF, text classification):

1. **Citation normalization** — regex + Chinese-numeral parsing → stable article keys.
2. **Ontology mapping** — high-frequency articles → 20 issue labels.
3. **Weak labeling** — citation-derived multi-label targets.
4. **Classification** — two contrasting methods (below).
5. **Evaluation & serving** — multi-label metrics + a Flask API + a Next.js demo.

Method A — Rule-based baseline (`baselines/rule.py`). Fires an issue when any of its ontology aliases/keywords appears in the scenario; confidence scales with the number of hits. Fully interpretable (it can point at the exact matched word), but brittle to unexpected phrasing.

Method B — TF-IDF + Linear SVM (`baselines/tfidf_svm.py`). Character n-gram (1–3) TF-IDF features → One-vs-Rest `LinearSVC` for multi-label output. It learns recurring character patterns rather than fixed keywords. In the live API, the SVM’s

`decision_function` is mapped through a sigmoid so the demo can show a real per-issue confidence (the 0.5 cutoff is exactly the SVM's native decision boundary).

*(BERT fine-tuning was scoped as optional future work and is intentionally **not** trained — the report and demo never fabricate BERT results.)*

---

5. Experiments and Testing

Two evaluations were run.

5.1 Held-out test set (229 rows, `eval/metrics.py`)

Standard multi-label metrics on the deterministic test split.

5.2 Acceptance suite (1,000 cases, `eval/acceptance_suite.py`)

Following the project's own acceptance spec: **50 cases × 20 labels = 1,000**, with each label split into **30 typical / 10 colloquial / 5 ambiguous / 5 near-miss** scenarios. Near-miss cases deliberately sit next to a confusable class (e.g. a 過失傷害 story phrased near 傷害; a 不當得利 误匯 case phrased near 詐欺). Pass threshold per label: Top-1 ≥ 0.95, recall ≥ 0.95, precision ≥ 0.90.

> **Honesty note:** this acceptance suite is deterministic and template-based. Although it has 1,000 cases, each label is cycled from only ~3–5 base phrasings, so its linguistic diversity is far lower than the count suggests. It is course-demo validation, **not** an independent human-labeled benchmark.

---

6. Results

6.1 Held-out test metrics

Method	Micro-F1	Macro-F1	Precision	Recall	Hamming Loss
Rule-based	0.600	**0.426**	0.470	**0.831**	0.062
TF-IDF + SVM	**0.760**	0.324	**0.851**	0.686	**0.025**

6.2 The interesting finding

I originally expected TF-IDF + SVM to win on every metric. **It did not.** TF-IDF + SVM clearly wins on Micro-F1 (0.76 vs 0.60), precision (0.85 vs 0.47), and Hamming loss — it is the better, more conservative classifier overall. But the **rule-based baseline wins Macro-F1 (0.43 vs 0.32)**.

The reason is class imbalance: TF-IDF + SVM scores **0.000 F1 on 9 of the 20 labels** (the rarest ones — `document_forgery`, `contract_breach`, `hit_and_run`, `intimidation`, etc.) because it never has enough examples to learn them, so it just never predicts them. The rule-based method still fires on those rare classes via keywords, so its *per-class* average is higher even though its overall quality is lower. This Micro-vs-Macro inversion is the single most important lesson of the project: **on an imbalanced multi-label task, headline accuracy hides the failure on the long tail.**

Per-label highlight (test F1): `酒駕/公共危險` Rule 0.94 / SVM 0.99 (huge support), vs `intimidation` Rule 0.56 / SVM 0.00, `hit_and_run` Rule 0.19 / SVM 0.00.

6.3 Acceptance suite (1,000 cases)

Metric	Value
Overall Top-1 accuracy	**0.800**
Overall Top-3 hit rate	**0.962**
Overall recall@5	**0.962**
Labels passing strict gate	1 / 20

By scenario difficulty:

Split	Cases	Top-1	Top-3	Recall@5
typical	600	0.857	1.000	1.000
colloquial	200	0.735	0.810	0.810
ambiguous	100	0.730	1.000	1.000
near_miss	100	0.600	1.000	1.000

Reading the numbers honestly: Top-3 / recall@5 are strong (0.96) — the correct issue is almost always in the candidate set. Top-1 drops on colloquial and near-miss cases, which is exactly where a keyword system struggles. Only 1/20 labels clears the deliberately strict per-label gate, because the gate demands ≥95% precision *and* recall on rare classes — a bar the current weak-label data cannot meet. This is reported, not hidden.

6.4 The solution / system delivered

- **20-issue ontology**, citation normalizer, weak-labeling and split pipeline (reproducible).
- **Two trained classifiers** with an honest comparison.
- **Flask API** (`/api/health`, `/api/predict`) returning labels, per-issue model grid, and **full statute text**.

- **Next.js + design-system frontend (LexTag)** — a bilingual “legal instrument” UI: scenario input, animated pipeline, ranked predictions with confidence bars, a 2-model comparison (line chart + matrix using real metrics), an explainability panel, a prominent *not-legal-advice* disclaimer, and — the key feature — **expandable cards showing the actual article text** for every supporting statute, not just the article number.

Live demo examples (real API output): a drink-driving story returns `酒駕/公共危險` + the full text of 刑法 §185-3; a Threads-impersonation story returns `妨害名譽` + the full text of 刑法 §309, §310 and 民法 §195.

---

7. Discussion

What worked. Framing the problem as *issue triage* (classification) instead of *legal QA* (generation) made it measurable, honest, and genuinely useful. The statute-text lookup turned “刑法 §185-3” from an opaque code into a verifiable clue. The two-model comparison surfaced a real, non-obvious ML lesson (Micro vs Macro under imbalance).

What was hard / where it fails. The rule-based + TF-IDF stack is fundamentally limited by (1) **weak-label noise**, (2) **class imbalance** — the long tail is starved of data, and (3) **colloquial phrasing** — keyword matching misses paraphrases like “撞了就跑” for 肇事逃逸 or “借了不還” for 所有物返還. These cases return no prediction and correctly trigger the “needs review” flag rather than guessing.

I was wrong about two things. First, I assumed the ML model would dominate the rule baseline everywhere — the Macro-F1 inversion proved otherwise. Second, I assumed more acceptance cases (1,000) implied more rigor — but because they cycle a few templates, the true linguistic coverage is modest, which is why the report says so explicitly.

Future work. Manual gold-label annotation for the rare classes; fine-tune `hfl/chinese-macbert-base` with a sigmoid multi-label head to capture implied semantics; add out-of-distribution / “no legal issue” detection.

---

8. Conclusion

LexTag is a reproducible Chinese-NLP pipeline that turns everyday legal narratives into multi-label issue predictions with citation-derived weak labels, compares a rule-based and a TF-IDF+SVM classifier with an honest analysis, and presents the result in an interactive demo that shows the **actual statute text** as a lookup clue. It is deliberately positioned as an upstream intake/routing aid — measurable, auditable, and free of legal-advice generation — which is precisely where it is most useful and least risky.

---

9. Reproducibility & Code

```
cd final_project
uv sync --extra dev
uv run python scripts/build_labels.py        # weak labels
uv run python scripts/split_data.py          # train/val/test
uv run python scripts/build_law_lookup.py    # statute text -> data/
        law_articles.json
uv run python baselines/rule.py              # rule predictions
uv run python baselines/tfidf_svm.py         # TF-IDF + SVM
uv run python eval/metrics.py                # report/results.md
uv run python eval/acceptance_suite.py       # 1,000-case acceptance
uv run pytest                                 # 20 tests
./start_demo.sh                               # Flask API + Next.js demo
```

Tests: **20 passed**. Pipeline and demo verified end-to-end.

---

10. References & Academic Integrity

All implementation is the author's own work. External datasets, models, and products are cited below.

Datasets

- Huang, Liang-Hsun. `lianghsun/tw-legal-synthetic-qa`. Hugging Face.
- Huang, Liang-Hsun. `lianghsun/tw-processed-law-article`. Hugging Face.

Libraries

- scikit-learn (TF-IDF, LinearSVC, metrics); jieba; Flask / flask-cors; PyYAML; Hugging Face `datasets`.
- Next.js, React, TypeScript (frontend); Noto Serif/Sans TC + IBM Plex Mono (typography).

Related products (competitive context)

- Lawsnote · Lawbot AI · LawChat · EasyLaw · LawAI (法詢).

Statute text is reproduced from public Republic of China (Taiwan) law via the `tw-processed-law-article` dataset, shown verbatim as a lookup clue only.